PS5 Debug & Cheat Tool — User Guide

Oct 12, 2026 · @sccs

An all-in-one PS5 debugging and cheat-making tool for ps5debug-NG: scan memory, follow pointers, call game functions, debug, log function calls, script in Python, and package everything into cheats you can turn on and off. This guide covers every tab and tool, with examples you can copy.

Quick start

You need a PS5 running ps5debug-NG 1.3 or newer, a Windows PC on the same network, and Python 3.11+.

1. Install the libraries once: `pip install keystone-engine capstone numpy`. Keystone assembles code, capstone disassembles it, numpy powers the pointer scanner.
2. Load ps5debug-NG on the console with your ELF loader. For the fastest pointer scans, load `ps5debug-NG-ptrscan.elf` from the `pointer map c` folder instead (see *The pointer-scan payload*).
3. Start the tool: `python gui.py`.
4. Type the console IP (e.g. `192.168.100.15`), keep port `744`, press **Connect**. The status dot turns green and the log shows the payload version, e.g. `ps5debug-NG by OSR v1.3.2 [ptrscan-2]`.
5. Start the game, press **Refresh** next to *Process*, and pick `eboot.bin`. The **PID** box fills in; every tab uses it.

Your first cheat in five minutes (a health value):

1. **Scanner** tab → Value type `4 Bytes`, Value `100` (your current HP) → **First Scan**.
2. Take damage in the game, type the new HP → **Next Scan**. Repeat until a few results remain.
3. Double-click the result to add it to the address list, click its **Active** box to freeze it.
4. Select the entry → **Copy as cheat JSON**, then in **Cheat Editor** select your game → **New cheat** and paste.
5. The address changes after a restart? Right-click it → **Pointer scan for this address…** and add a chain instead (see *Pointer scan*).

The window at a glance

The top bar holds the connection (IP, port, Connect, Ping, Diagnose, Force reset), the process picker with **Info** and **Maps**, the shared **PID** and the **Stack-safe calls** switch. Ctrl+1 to Ctrl+8 switch tabs.

Tab or tool	What it is for	Open with
Caller	Call game functions, manage remote buffers, assemble quick code	Ctrl+1
Scanner	Cheat Engine-style value scans, address list, freezing	Ctrl+2
Hex editor	View and edit memory, follow pointers, dump regions	Ctrl+3
Disassembler	Read code, patch, AOB signatures, code caves	Ctrl+4
Debugger	Breakpoints, watchpoints, stepping, registers, XMM	Ctrl+5
Cheats	Turn cheats on and off	Ctrl+6
Cheat Editor	Build, edit, import and export cheats	Ctrl+7
Kernel log	Live console log: crashes, faults, errors	Ctrl+8
Pointer scan	Find pointer chains to an address	Tools menu, or Scanner right-click
Function logger	Log every call of a function, game keeps running	Tools menu, or Disassembler/Debugger right-click
Python script	Script the game with the `ps5` API	Tools menu, or Ctrl+P

Errors appear in red on the status line and in the log (File → Open log window). Clicking while something runs queues the request instead of refusing it.

Caller tab

The Caller runs a game function on the console with arguments you choose and shows its return value (RAX). It follows the PS5 calling convention: **RIP** is the function, **RDI, RSI, RDX, RCX, R8, R9** are arguments 1 to 6.

Function call panel

- *Function address (RIP)* accepts any expression: `module(eboot.bin)+0x123450`, `@mybuf`, `$site`.
- Each argument row takes a number, an expression, or a buffer (`@name`). Tick **edit** to turn it into a cheat input when you record a cheat; pick an item list to make it a dropdown.
- **Call function** runs it; RAX appears with **Goto RAX** and **Use result as RDI**.

Remote buffers are memory you allocate inside the game: **New**, **Alloc**, **Free selected**, **Protect RW / RX / RWX**, **Copy pointer**, **Use as RDI**, **Open in hex**. Write text or bytes into the selected buffer with *Write contents into selected buffer*, with **append NUL** for strings.

Dereference / memory: *Follow ptr N times, then + offset* walks pointer chains; **Read / deref** shows the value as u64, u32, float and more; **Write primitive** writes one value.

Quick code (the **Assembler…** window): write x86-64, then **Assemble → set as RIP** (Ctrl+Enter) or **Assemble & Call now**. **auto ret** adds a missing `ret`; **auto stack frame** reserves stack when your code calls functions. Variables (`$count`) are typed in a table.

Example: give 50,000 souls by calling the game's add-souls function.

```
RIP  module(eboot.bin)+0x6A2F10
RDI  [module(eboot.bin)+0x3D1F28]      player object (pointer read)
RSI  50000
```

Example quick code that builds a 3-argument call needing stack space:

```
mov rdi, {@player}
mov esi, 0x2710        ; item id
mov edx, 1             ; quantity
mov rax, {module(eboot.bin)+0x6B0040}
call rax
```

Call safety (Settings → Remote call safety): every call borrows a game thread. *Stack-safe calls* runs it through a small trampoline on a separate, aligned stack. The *minimum gap between calls* (default 150 ms) stops rapid calls from crashing the game. Leave both on.

Scanner tab

The Scanner works like Cheat Engine, but the console does the scanning (turbo scan), so a first scan over 7 GB takes 2 to 3 seconds and only the hit count crosses the network.

Value types: Byte, 2/4/8 Bytes (tick *Signed* or *Hex*), Float, Double, String (tick *UTF-16* if needed), Array of bytes (`48 8B ?? 05` , `??` = any byte).

Scan type	First scan	Next scan
Exact value	yes	yes
Bigger than / Smaller than	yes	yes
Value between	yes (both bounds)	yes
Unknown initial value	yes (snapshot)	-
Increased / Decreased (by…)	-	yes
Changed / Unchanged	-	yes

Memory scan options: start and stop address, *Writable memory only*, *Include executable (code) regions*, *Skip slow / uncached regions (GPU)*, *Fast scan, alignment*, *Unknown value: include zeros*, *Fast read engine (aliasing)*, and for floats *Exact float compare* and *Simple values only*.

Results are paged 1,000 at a time with live values; a changed value turns red. Double-click a result to add it to the address list. Right-click for *Browse this memory*, *Find out what writes to / accesses this address* (adds a hardware watchpoint in the Debugger), breakpoints and *Pointer scan for this address…*.

Address list: click **Active** (or press Space) to freeze; double-click a description, address or value to edit it. Right-click: change value, change type, copy address, copy pointer expression. **Save table… / Load table…** keep lists in `tables/` . **Copy as cheat JSON** turns the selected entries into a cheat. Entries that came from a pointer scan show as `P→ address` and are followed again on every refresh.

Example: an unknown timer.

1. Value type `Float` , Scan type *Unknown initial value* → **First Scan**.
2. Wait a moment → *Decreased value* → **Next Scan**. Pause the game → *Unchanged value* → **Next Scan**.

3. Repeat until a handful remain, then freeze the right one.

Pointer scan

A pointer scan finds chains like `eboot.bin+3A1F28 → 18 → 240 → 1C4` that lead to your value from inside the game's module, so they still work after a restart. Open it from Tools → Pointer scan… or right-click an address in the Scanner.

Setting	Default	Meaning
Address to find	from the Scanner	The value's current address
Max level	5	How many pointers deep to look
Last offset	0x1000	How far into the object that holds the value (e.g. `+0x1C4`)
Other offsets	0x400	How far apart the pointers in between may be; keep it small
Max pointers / level	1,000,000	Safety limit; up to 8,000,000 with the pointer-scan payload
Base modules	all	Restrict where chains may start, e.g. `eboot`
Max results	50,000	Stop after this many chains

Modes

- *Targeted* (default): the console searches level by level and only the matches cross the network. With the pointer-scan payload each level is one pass of about 8 seconds over 7.5 GB.
- *Full pointer map*: builds a reusable map of every pointer (slow over the network) that you can save as `.ptrmap` and search offline.

Reading the result line: `pointers per level 519, 37,349, 850,843` means 519 places point to your address, 37,349 point to those, and so on. If a level hits the limit, lower *Other offsets* (0x200) or raise *Max pointers / level*.

Rescan narrows the list: restart the game or reload, find the value again, type its new address and press **Rescan**. Only chains that still reach it stay. Two or three rescans usually leave a few good chains.

Using a chain: double-click it to add it to the address list (green = still points to the address). Right-click → *Copy as cheat expression* gives `[[[module(eboot.bin)+0x3A1F28]+0x18]+0x240]+0x1C4`, ready for any cheat step. Results save as `.ptrs`.

Example: an HP value that moves every load.

1. Scan for HP until one address is left: `105C770610`.
2. Pointer scan, defaults → 40 chains.
3. Load a save, find HP again at `1061A20610` → **Rescan** → 3 chains.
4. Add the shortest one to the address list, freeze it, **Copy as cheat JSON**.

Hex editor

The Hex editor shows memory as hex and text and lets you type over it. Edits are held until you press **Write changes**; **Revert** drops them.

- **Address** accepts any expression: `@stats`, `[module(eboot.bin)+0x3D1F28]+0x40`, `$site`. **Goto RIP**, **Goto RAX**, **Goto deref** and **Goto selected buffer** jump to the obvious places.
- **Prev page / Next page** move by a screen; **Follow newest** keeps the view on the last place you jumped to.
- Select with drag, Shift+click or Shift+arrows. Ctrl+C copies hex, Ctrl+Shift+C the address; **Copy as…** gives hex with or without spaces, `\x48` escapes, a C array, ASCII text or a full hex dump.
- **Insert id at cursor** writes an item id from the chosen item list at the cursor.
- **Save selection / view…** writes bytes to a file; **Dump region from console…** saves a whole memory region.

Example: change a 32-bit gold value at a known offset.

1. Address `[module(eboot.bin)+0x3D1F28]+0x2A0` → Enter.
2. Click the first byte, type `3F 42 0F 00` (999,999 little-endian).
3. **Write changes**.

Disassembler

The Disassembler shows game code with calls, jumps and conditional jumps coloured, and module+offset labels. Type an address or expression and press Enter; **◀ Back / Forward ▶** retrace your steps; **Next page ▼** reads further. *Instructions* sets how many lines to read.

Right-click an instruction to:

- *Follow* a call, jump or RIP-relative operand.
- Toggle a breakpoint (F9), choose a breakpoint type, or *Run to here*.
- *NOP this instruction*, *Patch bytes…*, *Assemble here…*, and *Undo last patch*.
- *Code cave…*: replace the instruction with a jump to your own code (below).
- *Log calls to this function…* (see *Function logger*).
- *Find references to this address*, *Make AOB signature here*, *Open in hex editor*.

AOB bar: type a pattern such as `48 8B 05 ?? ?? ?? ?? 8B 40 1C` and press **Scan** to find it in the game's code. **Make AOB from selected line** builds the shortest unique pattern, wildcarding addresses that change between versions. **Copy as cheat step** gives an `aob` step so your cheat finds its site by pattern and survives game updates.

Code caves: the dialog shows the instructions being replaced and a cave editor. Write your code, check **Assemble preview**, then **Enable hook**; **Disable** restores the original bytes. *Locate by AOB* saves the cheat with an `aob` step instead of a fixed offset. **Save cheat** writes it as a toggle or once cheat.

Example cave: double all damage taken by the player (the instruction subtracts `eax` from HP).

```
; original: sub dword ptr [rbx+0x1C4], eax
cmp rbx, qword ptr [rip:@player]   ; only the player
jne skip
shr eax, 1                          ; halve the damage
skip:
sub dword ptr [rbx+0x1C4], eax
```

Debugger

The Debugger attaches to the game, stops it at breakpoints and watchpoints, and steps through code. Press **Attach** first; the console sends stop events to your PC on TCP port 755, so allow it through the firewall.

Breakpoint type	Stops when	Limit
Software (int3)	that instruction runs	28 at once
Hardware execute	that instruction runs, code not modified	4 hardware slots shared
Hardware write only	the address is written (1, 2, 4 or 8 bytes)	4 hardware slots shared
Hardware read/write	the address is read or written	4 hardware slots shared

Keys: F5 continue, F6 pause, F7 step into, F8 step over, Shift+F8 step out, F4 run to cursor, F9 toggle breakpoint on the selected line.

Panels

- *Registers*: double-click to edit, right-click to copy or view in the hex editor. Changed values turn red.
- *Call stack*: callers found by the console's stack walk, each checked for a real call before it.
- *Stack (RSP)*: raw stack with return addresses marked.
- *Threads*: every game thread; *Show registers*, *Suspend*, *Resume*.
- *XMM / FPU*: XMM0–15 or YMM (256-bit) as float ×4, double ×2, int32/int64 or bytes, plus MXCSR. Double-click to edit, e.g. type `1.5, 2` to set lanes 0 and 1.

Only the open panel loads on each stop, so stepping stays fast; the log shows how long each stop took, e.g. `(45 ms)`. When a breakpoint hits, the console has already run that instruction; register edits go to the live state.

Example: find what writes your HP.

1. Scanner → right-click the HP address → *Find out what writes to this address*.
2. Take damage: the game stops on the instruction after the write, e.g. `sub dword ptr [rbx+0x1C4], eax`.
3. Right-click that line → *Show in Disassembler tab*, then build a code cave or a NOP patch there.

Function logger

The function logger records every call of a function while the game keeps running at full speed, with no breakpoint. The first instructions of the function jump to a small recorder that saves the call into a ring buffer and then runs those instructions itself.

Open it by right-clicking an instruction in the Disassembler or Debugger → *Log calls to this function (no breakpoint)…*, or Tools → Function logger… and type the address (`module(eboot.bin)+0x6A2F10`, `$site`).

For each call you see the call number, time, the caller (return address), and arguments 1 to 6 (rdi, rsi, rdx, rcx, r8, r9) plus xmm0 as a float. Select a call to see every register, RFLAGS, RSP, XMM0–3 as floats and doubles, and stack arguments 7 to 11.

- **group by caller** shows one row per call site with a count: the fastest way to see who calls a function.
- **signed** shows negative numbers; **pause view** freezes the table while logging continues.
- Right-click: view any argument in the hex editor, show the caller in the Disassembler, copy the call.
- **Export CSV…** saves everything; **Stop** puts the original bytes back.

The ring holds 1,024 calls; for functions called thousands of times a second the status line counts calls it had to skip (*missed*).

Example: which item id does the game pass when you pick something up?

1. Find the add-item function (e.g. through a write watchpoint on your inventory count).
2. Log it, pick up an item in the game.
3. The new row shows `rsi = 2710` and `rdx = 1`: item id 0x2710, quantity 1. Use those as call arguments in a cheat.

Cheats tab

The Cheats tab is the everyday view: pick a game on the left, flip cheats on the right. The *ON* column shows how many cheats are on per game.

- **ON / OFF** toggles a toggle cheat; **Run** runs a one-shot cheat. ⟳ re-applies a cheat that is on (useful after a level load).
- Inputs sit on the same line. Item lists are searchable dropdowns; **All…** runs the cheat for every item in the list, several items per call, with retry and stop-at-first-error options.
- Hover the name or ⓘ to read the description at the bottom.
- The search box filters across all games and descriptions.
- **Find process** picks the game's PID by title id; **Turn all off** disables every toggle cheat of that game; *verify title id* refuses to run a cheat on the wrong game.

- **Import…**, **Reload** and **Python script…** are in the top bar.

Example: add every ring in the game. Select *Add Rings* → **All…** → *Items per remote call* 8 → **Start**. The progress shows each item; failures are listed at the end.

Cheat Editor

The Cheat Editor is where cheats are built. The left tree lists games (one `.json` file each in `cheats/`) and their cheats; the right side shows the selected cheat as JSON with run buttons and tools.

Games

- **New game** asks for name, title id and version, filled in from the selected game. If that title id already has a file, the date is added to the file and the name (`PPSA01341_2026-10-12.json`, "Demons Souls (2026-10-12)"), so you can keep several versions side by side. Tick *Copy the N cheats* to start from a copy.
- **Delete game** moves the file to `cheats/deleted/` with a timestamp; move it back to restore it. It warns if cheats are still on.

Cheats: **New cheat**, **Delete**, **Save edits** (after editing the JSON), **Run / Toggle**, **Apply (keep on)**, **Disable**, **Find game process**.

Building tools

- **Record cheat from Caller tab…** captures the function, arguments, buffers with their contents, quick-code variables and hex edits into one cheat. Registers ticked *edit* become inputs.
- **Insert Caller call step** adds the current Caller call (and every buffer and code it needs) to the selected cheat.
- **Insert code step…** opens an assembler with a live preview.
- **Python step…** edits the cheat's Python step in the script window; Ctrl+S writes it back.
- **Check & fix** finds `@buffers` used before any step creates them and adds the missing `alloc` / `code` steps from the current session; it also understands buffers made by `ps5.alloc` in Python steps.
- **Format help** shows the format reference below.

Import / export

- **Export…** saves the selected game or one cheat as a compact `.ps5c` file, including the item lists its inputs use. **Copy code** gives a `PS5C1:…` text code to paste in chat or forums; **Paste code…** imports one.
- **Import…** and **Import folder…** accept `.ps5c`, this tool's JSON, GoldHEN JSON, `.shn`, `.mc4` and decrypted ShnExt. Cheats merge into the matching game by title id: identical cheats are skipped, changed ones are added as "(imported)".
- Importing a file that contains Python steps asks first, because Python runs code on your PC.

Cheat format reference

A cheat file (`ps5cheat/1`) is JSON: one game, many cheats, each cheat a list of steps. `"mode": "toggle"` cheats have ON and OFF: patches are restored automatically when turned off, and optional `"off"` steps run then. `"mode": "once"` cheats just run.

```
{
  "format": "ps5cheat/1",
  "game": "Demons Souls", "title_id": "PPSA01341", "version": "01.004.000",
  "process": "eboot.bin", "author": "you",
  "cheats": [
    { "name": "Infinite HP", "mode": "toggle", "description": "HP never
drops",
      "steps": [ ], "off": [ ] }
  ]
}
```

Expressions work anywhere an address or value goes.

Write	Means
`0x10`, `123`, `-5`	numbers
`module(eboot.bin)`	base of a module (survives ASLR); `module(main)` = the game; `module(main#1)` = its 2nd section
`[ expr ]`	read the 8-byte pointer stored at expr
`@buf`	address of remote buffer *buf*
`$name`	a variable set by `read`, `call`, `aob`, `set`, or an input

Write	Means
`rax`	result of the last call
`+ - *`	arithmetic, e.g. `[[module(eboot.bin)+0x4A1C00]+0x20]+0x1C4`

Types: `u8 i8 u16 i16 u32 i32 u64 i64 ptr f32 f64 utf8 utf16 hex`.

Steps

Step	Example	What it does
write	`{"op":"write","addr":"$hp","type":"u32","value":999}`	write a value
patch	`{"op":"patch","addr":"$site","type":"hex","value":"90 90"}`	write and restore on OFF
read	`{"op":"read","addr":"@stats","type":"u32","store":"lvl"}`	read into `$lvl`
set	`{"op":"set","var":"x","value":"$lvl*2"}`	compute a variable
check	`{"op":"check","addr":"$site","type":"hex","value":"29 83 C4 01"}`	stop if bytes differ (wrong version)
alloc	`{"op":"alloc","name":"stats","size":64,"prot":"rw"}`	allocate buffer `@stats` (reused if it exists)
free	`{"op":"free","name":"stats"}`	free a buffer
code	`{"op":"code","name":"fn","asm":"lea rax, [rdi+rsi]\nret"}`	assemble into executable buffer `@fn`
call	`{"op":"call","rip":"@fn","args":[2,40],"store":"r"}`	call with up to 6 args
aob	`{"op":"aob","pattern":"29 83 C4 01 00 00","store":"site"}`	find code by pattern into `$site`
asmpatch	`{"op":"asmpatch","addr":"$site","asm":"nop\nnop"}`	assemble at the address, restore on OFF
hook	`{"op":"hook","name":"hk","addr":"$site","orig":"auto","length":6,"asm":"…"}`	code cave, restored on OFF

Step	Example	What it does
protect	`{"op":"protect","addr":"@fn","size":64,"prot":"rwx"}`	change memory protection
python	`{"op":"python","code":"ps5.write_u32('@stats', 99)"}`	run a Python script
log, sleep	`{"op":"log","text":"hp","value":"$hp"}`, `{"op":"sleep","ms":250}`	show text, wait
repeat	`{"op":"repeat","count":3,"delay_ms":100,"steps":[…]}`	run steps several times

Extra keys: `"near":"$site"` on alloc/code allocates within 2 GB (needed for RIP-relative code and 5-byte jumps); `"auto_frame": false` and `"auto_ret": false` on code; `"raw": true` on call skips the stack-safe trampoline; `"module"`, `"index"`, `"offset"`, `"data": true` on aob.

Inside asm, `{expr}` is replaced by the value of any expression (`{@stats}`, `{$item}`, `{module(eboot.bin)+0x10}`), and `[rip:expr]` makes a RIP-relative operand to that address.

Inputs appear in the Cheats tab and become `$name`:

```
"params": [
  { "name": "item", "label": "Item", "type": "u32", "default": "Moon Grass",
"items": "goods" },
  { "name": "qty", "label": "Quantity", "type": "u32", "default": 1 }
]
```

`"items"` names a list in `cheats/items/` (`{"Moon Grass": 1001, "Mega Sword": "0x2F3"}`), shown as a searchable dropdown with **All…**. Any `{playername}` inside a utf8/utf16 value becomes a text input automatically.

Complete cheat examples

Each example is one cheat object: paste it into the `cheats` list of a game file, or select a game in the Cheat Editor → **New cheat** and replace its JSON. Offsets and patterns are illustrations; use the ones you found for your game.

1. Set a value through a pointer chain (from a pointer scan)

```
{ "name": "Max gold", "mode": "once",
  "steps": [
    { "op": "write", "addr": "[[module(eboot.bin)+0x3D1F28]+0x18]+0x2A0",
"type": "u32", "value": 999999 }
  ] }
```

2. Version-proof NOP patch (finds the code by AOB, checks it, restores on OFF)

```
{ "name": "No HP loss", "mode": "toggle",
  "steps": [
    { "op": "aob", "pattern": "29 83 C4 01 00 00 8B 83", "store": "site" },
    { "op": "check", "addr": "$site", "type": "hex", "value": "29 83 C4 01 00
00" },
    { "op": "patch", "addr": "$site", "type": "hex", "value": "90 90 90 90 90
90" }
  ] }
```

3. Code cave: only the player is invincible (enemies still take damage)

```
{ "name": "God mode (player only)", "mode": "toggle",
  "steps": [
    { "op": "aob", "pattern": "29 83 C4 01 00 00", "store": "site" },
    { "op": "hook", "name": "god", "addr": "$site", "orig": "auto", "length":
6,
      "asm": "cmp rbx, qword ptr [rip:module(eboot.bin)+0x3D1F28]\nje
skip\nsub dword ptr [rbx+0x1C4], eax\nskip:" }
  ] }
```

4. Call a game function with an item dropdown

```
{ "name": "Add item", "mode": "once",
  "params": [
    { "name": "item", "label": "Item", "type": "u32", "default": "Moon
Grass", "items": "goods" },
    { "name": "qty", "label": "Quantity", "type": "u32", "default": 10 }
  ],
  "steps": [
    { "op": "call", "rip": "module(eboot.bin)+0x6B0040",
      "args": ["[module(eboot.bin)+0x3D1F28]", "$item", "$qty"] }
  ] }
```

5. Buffer + assembled code + call (all stats to 99)

```
{ "name": "All stats 99", "mode": "once",
  "steps": [
    { "op": "alloc", "name": "stats", "size": 64, "prot": "rw",
      "init": { "type": "hex", "value": "63 63 63 63 63 63 63 63" } },
    { "op": "code", "name": "setstats",
      "asm": "mov rdi, [{module(eboot.bin)+0x3D1F28}]\nmov rsi, {@stats}\nmov
edx, 8\nmov rax, {module(eboot.bin)+0x6C1200}\ncall rax" },
    { "op": "call", "rip": "@setstats" }
  ] }
```

6. Text input (the box *playername* appears automatically)

```
{ "name": "Change player name", "mode": "once",
  "steps": [
    { "op": "alloc", "name": "pname", "size": 32, "prot": "rw",
      "init": { "type": "utf16", "value": "{playername}" } },
    { "op": "call", "rip": "module(eboot.bin)+0x5F00A0", "args": ["
[module(eboot.bin)+0x3D1F28]", "@pname"] }
  ] }
```

7. Python step (loops are easier in Python)

```
{ "name": "Refill all consumables", "mode": "once",
  "steps": [
    { "op": "python", "code": "inv = ps5.pointer(ps5.module() + 0x3D1F28,
0x80)\nfor slot in range(64):\n    item = inv + slot * 0x10\n    if
ps5.read_u32(item) != 0:\n        ps5.write_u32(item + 4, 99)" }
  ] }
```

8. Toggle with off steps (turn a debug flag on, and off again)

```
{ "name": "Debug camera", "mode": "toggle",
  "steps": [ { "op": "write", "addr": "module(eboot.bin)+0x41A0C8", "type":
"u8", "value": 1 } ],
  "off":   [ { "op": "write", "addr": "module(eboot.bin)+0x41A0C8", "type":
"u8", "value": 0 } ] }
```

Python scripting

Scripts drive the game through the `ps5` object (also called `game`). Open Tools → Python script… (Ctrl+P): F5 runs, Esc stops, errors highlight the script line. The API list on the right inserts a call on double-click. **Save into cheat…** turns the script into a cheat; **Save .py… / Open .py…** use the `scripts/` folder; **Undo script patches** restores what scripts patched.

Scripts share everything with JSON steps: `ps5.alloc("stats")` creates the same `@stats` that steps use, `$variables` are `ps5.var(...)`, and patches made with `ps5.patch(...)` or `undo=True` are restored when a toggle cheat is turned off. Addresses may be numbers or any cheat expression.

Area	Calls
Read	`read(addr, size)`, `read_u32(addr)` (also u8 u16 u64 i8 i16 i32 i64 f32 f64 ptr), `read_values(addr, count, "u32")`, `read_string(addr, 64, "utf-16")`, `read_multi([(addr, size), …])`
Write	`write(addr, b"…" or "90 90")`, `write_u32(addr, v, undo=False)`, `write_string(addr, "text")`, `fill(addr, size, 0)`, `patch(addr, data)`
Pointers	`pointer(base, 0x20, 0x1C4)` = `[[base]+0x20]+0x1C4`, `addr("[module(eboot.bin)+0x10]+8")`
Buffers	`alloc("name", 0x1000, init=b"" or "text", near=None)`, `buffer("name")`, `buffers()`, `free("name")`
Code	`asm("mov eax,1\nret", at=0)`, `code("name", asm)`, `asm_patch(addr, asm)`, `nop(addr, n)`, `hook(addr, asm, name="h", length=None)`, `protect(addr, size, "rwx")`
Calls	`call(func, a1 … a6)` returns RAX; `f32(1.5)` gives float bits

Area	Calls
Find	`module("eboot.bin")`, `maps()`, `aob("48 8B ?? 05", module="main", index=1)`, `aob_all(pattern, limit=32)`, `find_value(99, "u32", start, length, count)`, `find_bytes("AA ?? BB", start, length, count)`
Cheats	`var("x")` / `var("x", 5)`, `param("amount")`, `step({…})`, `steps([…])`, `g` (dict kept between runs)
Control	`log(…)`, `sleep(s)`, `stopped()`, `wait_until(fn, timeout=10)`, `every(seconds, fn, times=None)`

`find_value` without a start address searches all game data on the console (fast turbo scan); with `start` and `length` it searches only that range. `struct`, `math`, `time` and `re` are already imported.

Example: pointer chain and a frozen value

```
hp = ps5.pointer(ps5.module() + 0x3D1F28, 0x18, 0x1C4)
ps5.log("HP now", ps5.read_u32(hp))
ps5.every(0.2, lambda: ps5.write_u32(hp, 9999))   # until you press Stop
```

Example: find code by AOB and patch it (undone when the cheat is turned off)

```
site = ps5.aob("29 83 C4 01 00 00 8B 83")
if site is None:
    raise RuntimeError("pattern not found - different game version?")
ps5.nop(site, 6)
ps5.var("site", site)            # later JSON steps can use $site
```

Example: find a value near a structure

```
player = ps5.pointer(ps5.module() + 0x3D1F28)
for a in ps5.find_value(42, "u32", start=player, length=0x800):
    ps5.log("42 at offset", hex(a - player))
```

Example: allocate, assemble and call

```
buf = ps5.alloc("stats", 64, init=bytes([99]) * 8)
fn = ps5.code("setstats", f"""
mov rdi, [{ps5.module() + 0x3D1F28:#x}]
mov rsi, {buf:#x}
mov edx, 8
mov rax, {ps5.module() + 0x6C1200:#x}
call rax
""")
ps5.log("result", ps5.call(fn))
```

Example: give every item in a list, with a pause between calls

```
add_item = ps5.module() + 0x6B0040
player = ps5.pointer(ps5.module() + 0x3D1F28)
for item_id in range(0x2710, 0x2720):
    ps5.call(add_item, player, item_id, 1)
    ps5.sleep(0.2)
```

Example: wait for a game state, then act

```
loading = ps5.module() + 0x41B000
ps5.wait_until(lambda: ps5.read_u8(loading) == 0, timeout=30)
ps5.write_f32(ps5.pointer(ps5.module() + 0x3D1F28, 0x2F0), 2.0)   # run speed
x2
```

Example: remember a value when turned on, restore it when turned off (a toggle cheat with a Python step in *steps* and one in *off*)

```
# steps
speed = ps5.pointer(ps5.module() + 0x3D1F28, 0x2F0)
ps5.g["old_speed"] = ps5.read_f32(speed)
ps5.write_f32(speed, 3.0)
# off
ps5.write_f32(ps5.pointer(ps5.module() + 0x3D1F28, 0x2F0),
ps5.g.get("old_speed", 1.0))
```

Python steps can run any code on your PC, so the Cheat Player never runs them and importing a file with Python steps asks first.

Kernel log

The Kernel log tab streams the console's `/dev/klog` from TCP port 3232, on its own connection so it never slows other tabs. It is the first place to look when a call, hook or cheat makes the game close: crashes, page faults, signals and errors show in red, warnings in amber.

- *connect with the console* (on by default) starts it when you press Connect; it reconnects by itself.
- Filter by text or `/regex/`, tick *errors only*, *time*, *auto-scroll* or *pause* (nothing is lost while paused).
- *errors to main log* copies error lines into the main log.
- **Save…** writes the last 20,000 lines to a text file; **Clear** empties the view.

Example: after a crash, type `/eboot|fault|signal/` in the filter to see only the lines about the game.

Other cheat formats

The Cheat Editor's **Import…** reads these formats and converts them into this tool's cheats.

Format	Extension	Notes
This tool, compact	`.ps5c`, or a `PS5C1:…` text code	About 4× smaller than JSON; includes the item lists the cheats use
This tool, full	`.json` (`ps5cheat/1` or `ps5pack/1`)	The files in `cheats/`
GoldHEN	`.json`	Offset patches become `patch` steps
SHN	`.shn`	XML trainer; padding after `</Trainer>` is ignored
MC4	`.mc4`	An encrypted SHN, decrypted with the public key
Decrypted ShnExt	`.ProjReap` folder or `.ReapPatch` files	AOB variables become `aob` steps, code caves become `hook` steps, data variables become buffers or inputs

Encrypted `.ShnExt` files are only opened by the Cheat Player (see below); the main tool does not include that decryptor.

Example: share a cheat on a forum. Select it → **Copy code** → paste the `PS5C1:…` line. Whoever receives it uses **Paste code…**.

Cheat Player

The Cheat Player (`cheat_player/`) is a small separate app for people who only want to play cheats: connect, pick a game, flip switches. It uses the same cheat engine as the main tool.

- **+** adds cheat files, 📁 loads every cheat in a folder and its sub-folders, **−** removes a game from the library. Copies are kept in its `library/` folder.
- It reads every format above, plus encrypted `.ShnExt` files, decrypted in memory only; nothing decrypted is written to disk.
- It finds the running game automatically and marks its title in the list.
- Cheats with Python steps are shown with a **Python** tag and cannot be run there: the player is meant for other people's cheats, and Python runs code on your PC.

Build the single-file `PS5 Cheat Player.exe` with `cheat_player\build_player.bat` (needs `pip install pyinstaller pycryptodome keystone-engine`). Rebuild it after updating the main tool, because it bundles the shared engine files.

The pointer-scan payload

`ps5debug-NG-ptrscan.elf` is ps5debug-NG 1.3.2 with one extra command (`0xBDAACC30`): one pass over the game's memory per pointer-scan level, with only the matches sent back. Everything else is unchanged, and the tool still works with the original payload (the pointer scan then uses slower turbo scans).

1. Reboot the PS5 so the old payload is not running.
2. Load `pointer map c\ps5debug-NG-ptrscan.elf` with your ELF loader.
3. Connect: the log shows `[ptrscan-2]` after the version.

If the console ever refuses a pointer-scan request, the Kernel log shows a line starting `[ps5debug-NG ptrscan]` with the reason, and the scan falls back to turbo scans. *fast memory access (aliasing)* reads memory at full speed; untick it if the console ever crashes during a scan.

Rebuilding needs clang/LLVM (WSL Ubuntu works), not the Microsoft compiler. The source changes are in `pointer map c\ps5debug-NG-master` (new `ptr_scan.c`, `ptr_scan_core.c`, `ptr_scan.h`; small edits to `proc.c` and `scan_turbo.c`), and `ptrscan.patch` re-applies them to a newer ps5debug-NG with `git apply ptrscan.patch`.

Settings, files and folders

The tool remembers its IP, port, theme, window sizes, call safety and the last options of each window in `settings.json` (written when you close the tool). View → Light or Dark theme applies on the next start.

Path	Holds
`cheats/*.json`	One game per file, with its cheats
`cheats/items/*.json`	Item lists for dropdown inputs, e.g. `{"Moon Grass": 1001}`
`cheats/deleted/`	Games removed with Delete game
`tables/`	Scanner address lists
`pointerscans/`	Pointer maps (`.ptrmap`) and results (`.ptrs`)
`scripts/`	Python scripts (`.py`)
`pointer map c/`	The pointer-scan payload, its source and patch
`cheat_player/`	The Cheat Player app and its build script
`backup*/`	Earlier versions of the tool's files

Network ports: 744 commands (PC → PS5), 755 debugger events (PS5 → PC, allow in the firewall), 3232 kernel log (PC → PS5).

Troubleshooting

You see	What to do
Connection lost / status dot red	Press Connect again. A command was cut off mid-way, so the tool closed the connection rather than misread the next reply.
`call failed: ERROR`	The Debugger is holding the game stopped: press Continue (F5). Otherwise wait a moment and call once more.
The game crashes on calls	Keep *Stack-safe calls* on and the minimum gap at 150 ms or more; check the Kernel log for the faulting address.
`buffer @x is used but no step creates it`	Cheat Editor → **Check & fix**, or add an `alloc` step named `x`.
`AOB not found`	Different game version, or the code is already patched by another cheat. Make a new signature in the Disassembler.
`bytes at 0x… are […] but the cheat expects […]`	Wrong game version, a breakpoint, or another patch is on that spot.
Pointer scan: `stopped after level N`	Lower *Other offsets* (0x200) or raise *Max pointers / level*.
Pointer scan finds 0 chains	Raise *Max level*, or check *Base modules* is empty. Make sure the address is the value's current address.
Debugger: `no stop event came back`	Allow TCP port 755 in the Windows firewall.
Python step: `this app does not run`	You are in the Cheat Player; run that cheat in the main tool.
Python error with `(script line N)`	The script window highlights line N.
Function logger: `cannot hook`	The function is too short or another hook/breakpoint is there; pick the instruction after the prologue.

You see	What to do
Anything slow	File → Open log window shows timings; Connection → Diagnose tests the link. A cable is much faster than Wi-Fi.